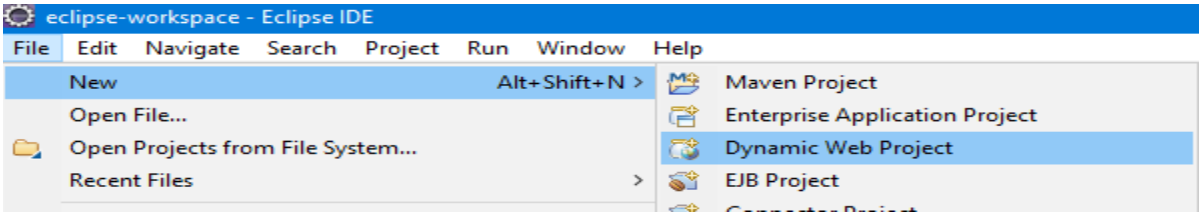
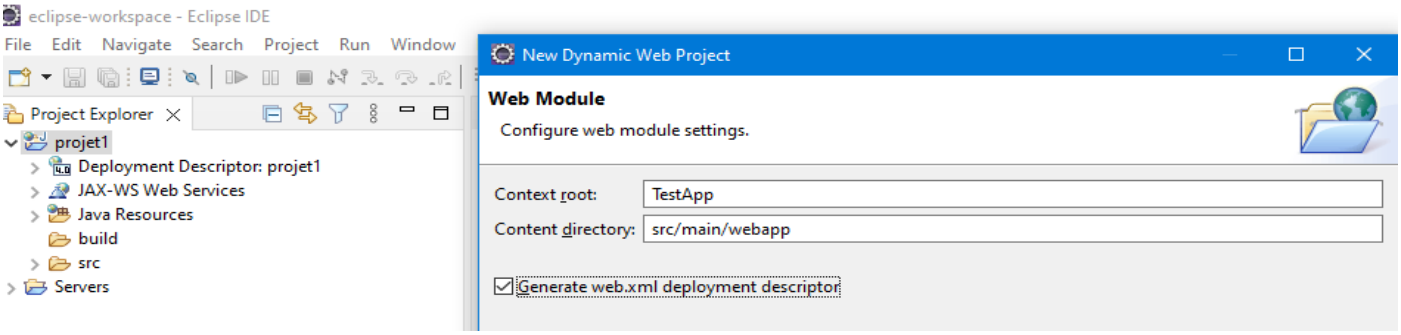
TP N°1

Redirection vers une page HTML

Etape1 : création d'un projet web dynamique

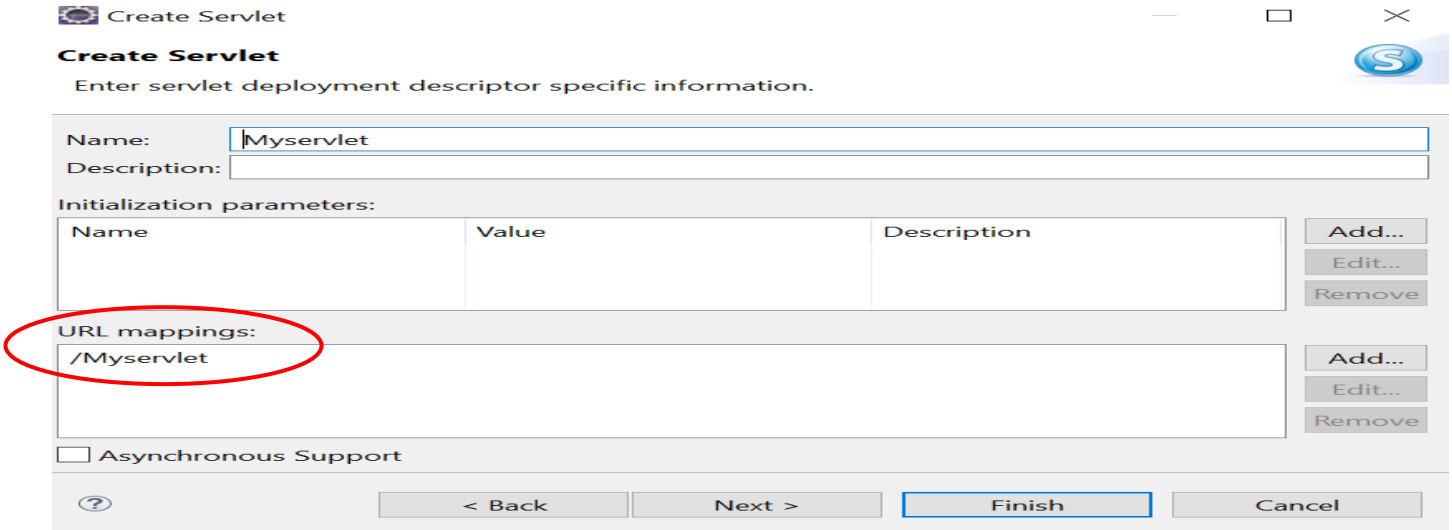


Sélectionnez web.xml, qui est un fichier de configuration qui se créer dans le dossier WEB INF

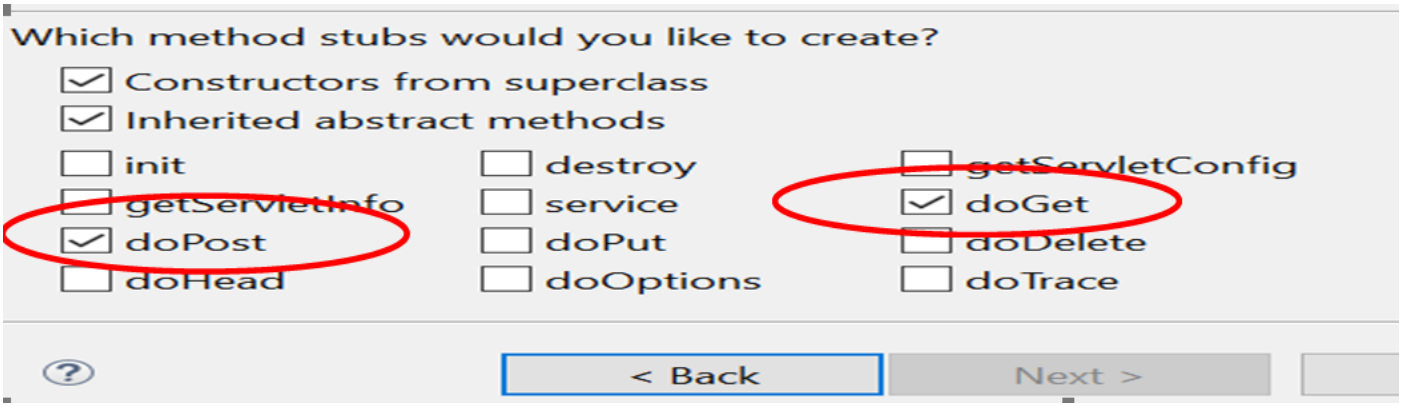


Etape 2 : création d'une servlet

Clic droit sur le projet => new => servlet=> donner un nom a la servlet exemple : Myservlet => next

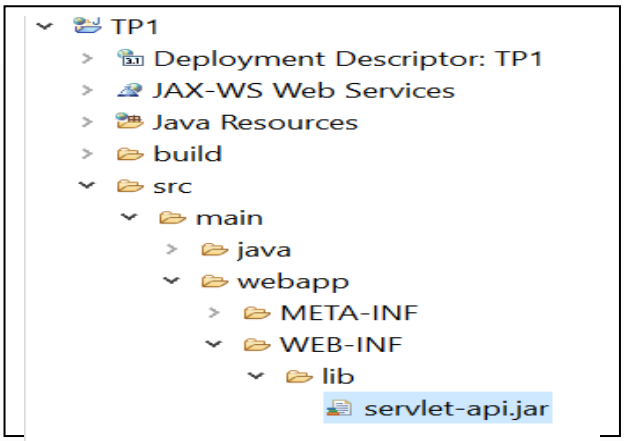


Next => puis laissez séléctionner les methodes doGet et doPost :



L'ouverture du servlet, on remaeque il y aurait des erreurs, il faut télécharger et coller le jar de servlet-api(le zip)

Toujours dans le dossier Lib du WEB INF coller le jar du sevlet-api



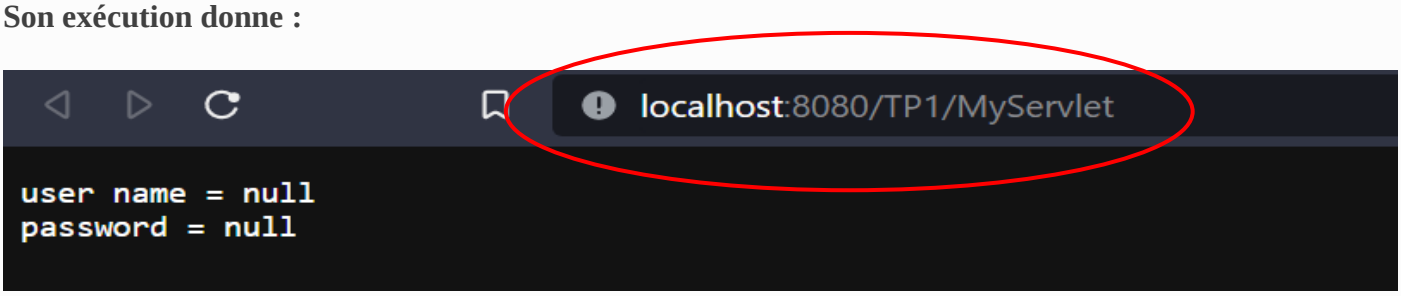
Etape 3 : Code de la servlet :

PrintWriter getWriter () : Cette méthode retourne un objet **PrintWriter** qui peut envoyer toute sorte de fichiers (texte, html, etc) au client.

MyServlet.java × user.html

```
2
3⊕import java.io.IOException;
11
12 @WebServlet("/MyServlet")
13 public class MyServlet extends HttpServlet {
14     private static final long serialVersionUID = 1L;
15
16
17     public MyServlet() {
18         super();
19         // TODO Auto-generated constructor stub
20     }
21
22
23     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOExcepti
24         PrintWriter out= response.getWriter();
25         out.println("user name = "+request.getParameter("user name"));
26
27         out.println("password = "+request.getParameter("password"));
28     }
```

Son exécution donne :



L'adresse ip de la servlet= l'@du serveur(localhost+port)/projet/Nomservlet

Structure d'une servlet HTTP

1. Une servlet HTTP est une classe Java qui hérite de la classe javax.servlet.http.HttpServlet

2. Par défaut, la classe javax.servlet.http.HttpServlet fournit des méthodes doXXX (XXX représentant une méthode HTTP) qui seront appelées lorsque la servlet devra traiter une requête HTTP de la méthode correspondante.

HttpServlet dispose donc des méthodes doGet, doPost, doPut… L'implémentation par défaut de ces méthodes consiste à retourner un message d'erreur HTTP. Chaque servlet doit donc redéfinir les méthodes qui la concernent.

Les méthodes doXXX ont toutes deux paramètres : javax.servlet.http.HttpServletResponse et javax.servlet.http.HttpServletRequest qui représentent respectivement la requête HTTP entrante et la réponse renvoyée par le serveur.

3. **java.io.PrintWriter HttpServletResponse.getWriter()**

Retourne un objet de type PrintWriter qui permet d'écrire la réponse dans le flux de sortie. L'objet PrintWriter offre des **méthodes write** pour générer une réponse au format texte (comme une page HTML).

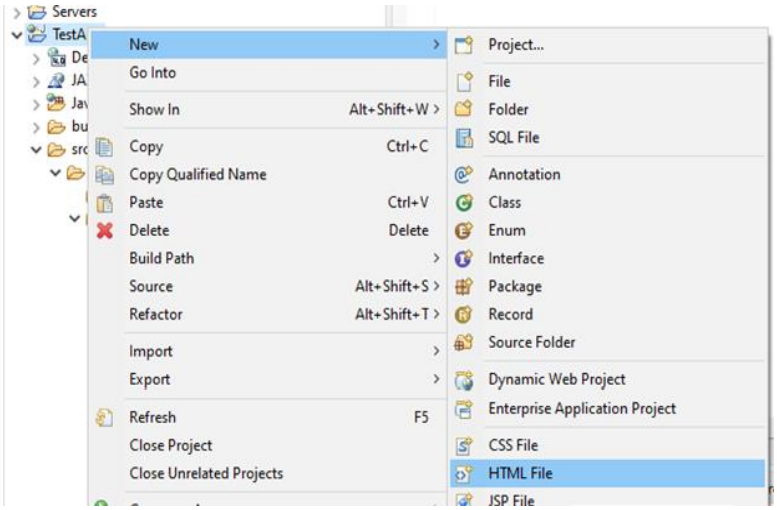
La méthode getParameter() de l'objet HttpServletRequest, prend en paramètre les données entrantes d'une autre page (exemple les données saisies par l'utilisateur) .

L'interface ServletResponse définit plusieurs méthodes qui permettent de fournir la réponse faite par la servlet suite à ses traitements :

Méthode	Rôle
SetContentType	Permet de préciser le type MIME de la réponse
ServletOutputStream getOutputStream()	Permet d'obtenir un flux pour envoyer la réponse
PrintWriter getWriter()	Permet d'obtenir un flux pour envoyer la réponse

Ici on copie l'adresse du servlet : elle contient l'@ip du serveur ici c'est localhost :port /Nom projet /nom servlet

<u>Etape 4 :</u> création d'une page html



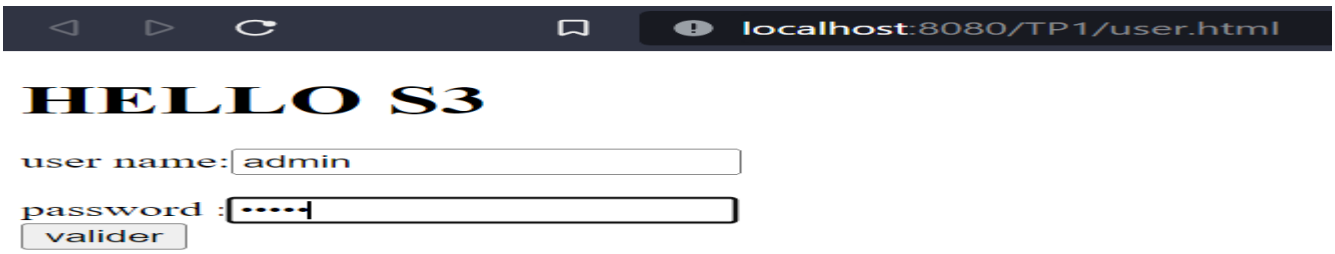
Clic droit sur le projet => new => HTML File => exemple : user.html

Insérez : - un titre hello S3
- Deux input user name et password
- Un bouton valider

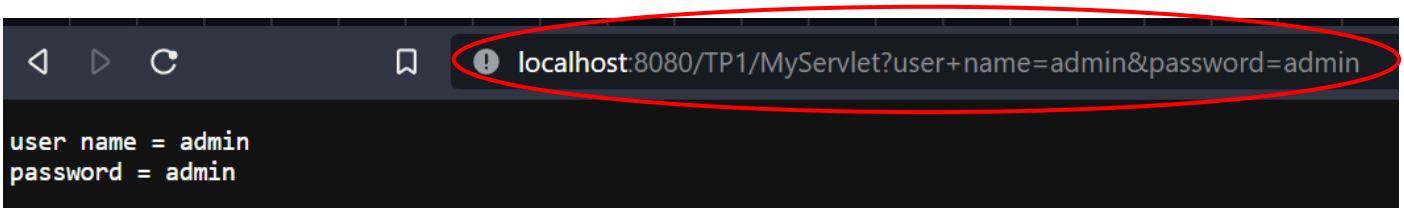
MyServlet.java | user.html

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<h1>HELLO S3 </h1>
<form action="http://localhost:8080/TP1/MyServlet" >

user name:<input type ="test" name ="user name"/><br>
<br>
password :<input type ="password" name="password"/><br>

<input type="submit" value=valider>
</form>

</body>
</html>
```

Après exécution de la page HTML :



Si on valide, les informations vont être dirigées vers la servlet :



Problèmes :

1. url trop longue

2. Le problème Les informations transitent dans l'URL, même le mot de passe et sont visibles, c'est critique car si on hake serveur d'application on retrouve tous les informations des utilisateurs, pour remédier à ce problème on utilise la méthode doPost pour récupérer des données des utilisateurs

<u>Etape 5 :</u> on insère ce code dans doPost

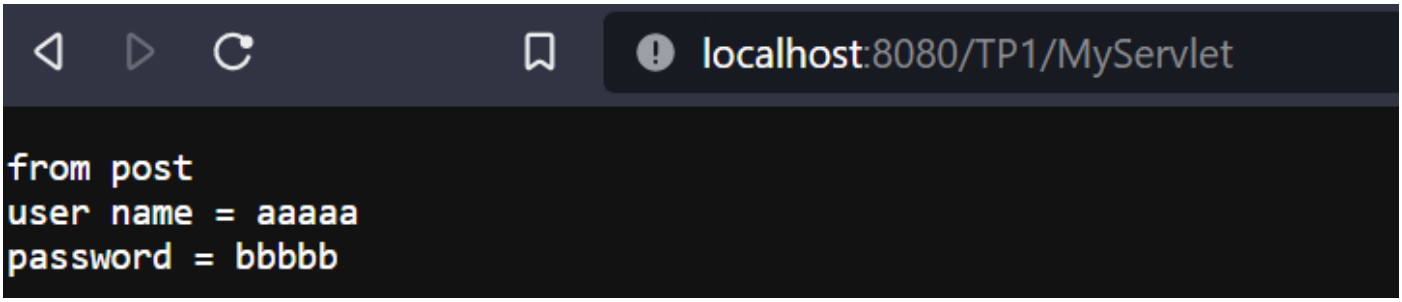
```
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOExcept
PrintWriter out= response.getWriter();
        out.println("from post");
        out.println("user name = "+request.getParameter("user name"));

        out.println("password = "+request.getParameter("password"));
    }
```

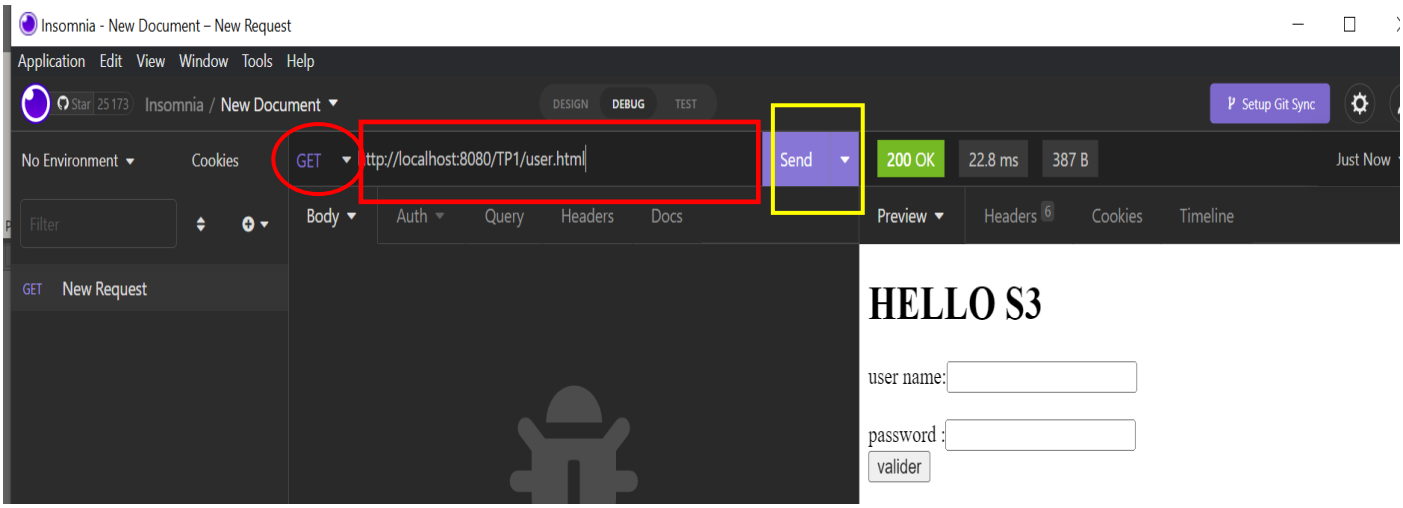
Dans la page Html on ajoute la méthode Post

MyServlet.java user.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="ISO-8859-1">
5  <title>Insert title here</title>
6  </head>
7  <body>
8  <h1>HELLO S3 </h1>
9  <form method="Post" action="http://localhost:8080/TP1/MyServlet" >
10
11 user name:<input type ="test" name ="user name"/><br>
12 <br>
13 password :<input type ="password" name="password"/><br>
14
15 <input type="submit" value=valider>
16 </form>
17
18 </body>
19 </html>
```

L'exécution (url clean) :



Etape Bonus : utilisation des outils de tests pour API (Postman ou Insomnia)

1. Insérez l'url de la servlet
2. Sélectionnez les méthodes
3. Send : pour voir le résultat de l'exécution